A

Project Report on

UNIT TESTING

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

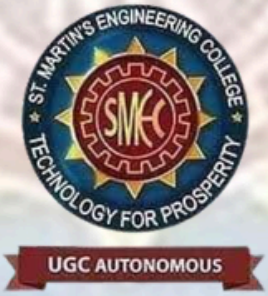
By

R.SUMAN -24K81A05H0

Under the guidance of

Mr. M. Dileep Kumar

Assistant professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St.MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

Certificate

This is to certify that the project entitled "**UNIT TESTING**" is being submitted by **R. SUMAN (24K81A05H0)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of Bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Project Internal Examiner
Mr. M. Dileep Kumar
Assistant professor
Department of CSE

Signature of HOD
Mr. D. KRISHNA KISHORE
Associate Professor and HOD
Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

SHAPINGVIKSIT BHARAT'SBRIGHTESTMINDS

UGC Autonomous

NBA& NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of "Bachelor of Technology in Department of Computer Science and Engineering", session: 2024–2028, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work **UNIT TESTING** is the outcome of my own bonafide work and is correct to the best of my knowledge, and this work has been undertaken taking care of Engineering Ethics. The result embodied in this project report has not been submitted to any university for the award of any degree.

R.SUMAN-24K81A05H0

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

We especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. Ravindra**, Director & Principal, St. Martin's Engineering College, Dhulapally, for permitting us to undertake this project

.

We are also thankful to **Dr. D. Krishna Kishore**, Head of the Department, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project as well as Project Coordinator **Mr. M. Dileep Kumar**, Assistant Professor, Department of Computer Science and Engineering, for his valuable support.

We would like to express our sincere gratitude and indebtedness to our project supervisor **Mr. M. Dileep Kumar**, Assistant Professor, Department of Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, we express thanks to all those who have helped us successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. We express thanks to all those who have helped us in successfully completing the project.

R.SUMAN-24K81A05H0

CONTENTS

1. ABSTRACT	**1**
2. INTRODUCTION	**2**
3. LITERATURESURVEY	**3**
4. SYSTEM REQUIREMENT	**5**
5. FLOW CHART	**6**
6. ALGORITHM	**7**
7. SOURCE CODE	**10**
8. EXPERIMENTALRESULTS	**11**
9. CONCLUSION	**19**
10.FUTURE ENHANCEMENT	**20**
11.REFERENCES	**21**

1. ABSTRACT

Unit testing is the practice of verifying the smallest testable parts of an application, such as functions and methods, in isolation. When it is combined with DevOps, these tests run automatically every time a developer commits code, so defects are detected within minutes rather than late in the release cycle. This project demonstrates Unit Testing using DevOps: a small Python application (a calculator module) is covered by automated unit tests written with the unittest framework, stored in a Git repository, and validated by a Continuous Integration (CI) pipeline built with GitHub Actions. The pipeline checks out the code, installs dependencies, runs all unit tests, and builds a Docker image only if every test passes. In the experiment, all 8 test cases passed. The project shows how automated testing inside a DevOps pipeline improves code quality, shortens feedback time, and makes software delivery faster and more reliable.

2. INTRODUCTION

Software today is released more frequently than ever, and manual testing cannot keep up with this speed. A bug that reaches production can cause financial loss, security problems, and loss of user trust. DevOps addresses this by joining development and operations teams and automating the build, test, and deployment stages. Within this approach, unit testing is the first and fastest quality gate: each function is tested on its own, using known inputs and expected outputs, to confirm that it behaves correctly. This project, Unit Testing using DevOps, builds a simple application and attaches an automated test suite to it. A Git repository stores the source code, and a CI pipeline runs the tests on every push or pull request. If any test fails, the pipeline stops and the developer is notified, so faulty code is never deployed. The project therefore shows the complete path from writing code and tests to automatic verification and containerised delivery.

ST. TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

3. L ITERATURE SURVEY

Software testing has long been recognised as a critical part of the development life cycle. Traditional approaches placed testing at the end of the project, after development was complete. Studies of such models show that defects found late are far more expensive to fix than defects found early. This observation led to test-driven development and the 'shift-left' idea, in which testing starts alongside coding. Unit testing frameworks such as JUnit, NUnit, unittest, and pytest give developers a standard way to write repeatable tests that check the behaviour of individual functions.

The DevOps movement extended this idea by introducing Continuous Integration and Continuous Delivery (CI/CD). In CI, developers merge their changes into a shared repository frequently, and an automated server builds the code and runs the test suite on every change. Tools such as Jenkins, GitHub Actions, GitLab CI, and Azure DevOps are widely used for this purpose. Existing research and industry reports show that teams using automated CI pipelines deploy more often, recover from failures faster, and have fewer defects reaching production. Containers, such as those built with Docker, further help by providing the same environment for testing on every machine.

Based on this background, the proposed project combines a unit testing framework, a version control system, a CI pipeline, and Docker into one simple workflow. Rather than relying on manual test execution, every commit is validated automatically, and the results are reported immediately. This makes the approach suitable for students and small teams who want a clear, practical example of how unit testing fits into a DevOps pipeline.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

4. SYSTEM REQUIREMENT

System Requirements

Hardware Requirements:

- Processor: Intel Corei3or equivalent
- RAM: 4 GB or above
- Storage: 10 GB freespace
- Internet connectionforGitHub and Docker Hub

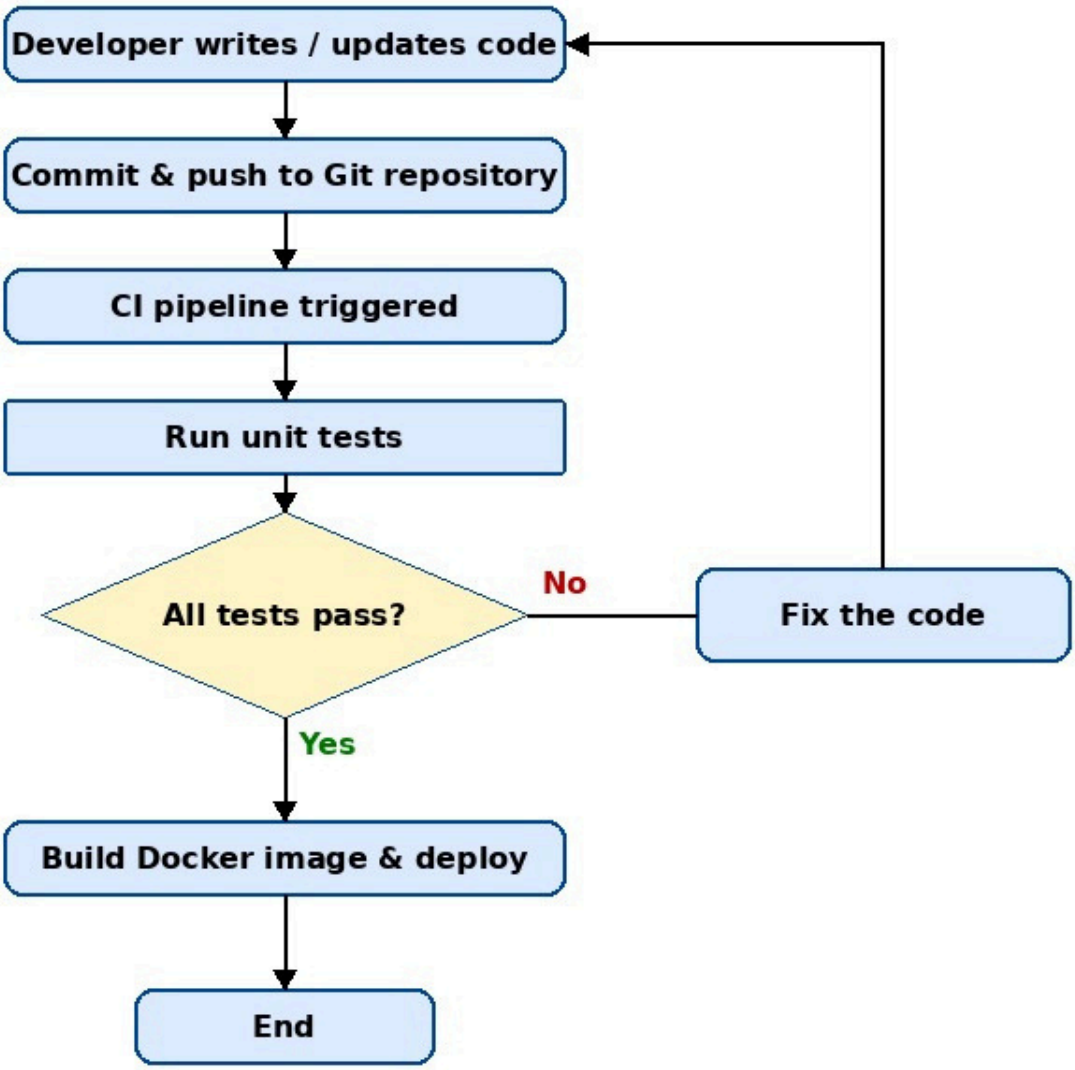
Software Requirements:

- Operating System:Windows/Linux/macOS
- Python 3.8 or above(unittest framework)
- Git and a GitHub account
- GitHub Actions (CI/CD) and Docker
- VS Code or any codeeditor

Functional Requirements:

- Write unit tests for application functions
- Run tests automatically on every push or pull request
- Display pass/fail test results
- Build a Docker image only when all tests pass
- Notify the developer when a test fails

5.FLOW CHART


Developer writes / updates code
Commit & push to Git repository
CI pipeline triggered
Run unit tests
All tests pass?
No
Fix the code
Yes
Build Docker image & deploy
End

6. ALGORITHM

1. **Start the process.**
2. Developer writesorupdatesthe application code.
3. Developer writesunittestcases for each function.
4. Commit and pushthecodetothe Git repository.
5. The CI pipeline istriggeredautomatically.
6. The pipeline checksoutthecode and sets up Python.
7. Run all unit testsusingtheunittest framework.
8. If all tests pass,buildtheDocker image and deploy.
9. If any test fails,stopthepipeline and notify the developer to fix the code.
10. **Stop the process.**

7.SOURCE CODE

```
# calculator.py
"""Simplecalculator module used as the application under test."""


def add(a, b):
    return a + b


defsubtract(a,b):
    return a - b


defmultiply(a,b):
    return a * b


def divide(a, b):
    if b == 0:
        raiseValueError("Cannot divide by zero")
    return a / b


def is_even(n):
    returnn%2== 0


def factorial(n):
    ifn<0:
        raiseValueError("Negative numbers are not allowed")
    result = 1
    foriinrange(2, n + 1):
        result *= i
    return result

#tests/test_calculator.py
import unittest
fromcalculatorimport add, subtract, multiply, divide, is_even, factorial


classTestCalculator(unittest.TestCase):

    deftest_add(self):
        self.assertEqual(add(2, 3), 5)
        self.assertEqual(add(-1, 1), 0)

    deftest_subtract(self):
        self.assertEqual(subtract(5, 3), 2)
        self.assertEqual(subtract(3, 5), -2)
```

```
    deftest_multiply(self):
        self.assertEqual(multiply(4, 3), 12)
        self.assertEqual(multiply(5, 0), 0)

    deftest_divide(self):
        self.assertEqual(divide(10, 2), 5)
        self.assertAlmostEqual(divide(1, 3), 0.3333, places=3)

    deftest_divide_by_zero(self):
        withself.assertRaises(ValueError):
            divide(5, 0)

    deftest_is_even(self):
        self.assertTrue(is_even(4))
        self.assertFalse(is_even(7))

    deftest_factorial(self):
        self.assertEqual(factorial(0), 1)
        self.assertEqual(factorial(5), 120)

    deftest_factorial_negative(self):
        withself.assertRaises(ValueError):
            factorial(-3)


if__name__ == "__main__":
    unittest.main()

#Dockerfile
FROMpython:3.11-slim
WORKDIR /app
COPY ..
CMD["python", "-m", "unittest", "discover", "-s", "tests", "-t", ".", "-v"]
```

8. EXPERIMENTAL RESULTS

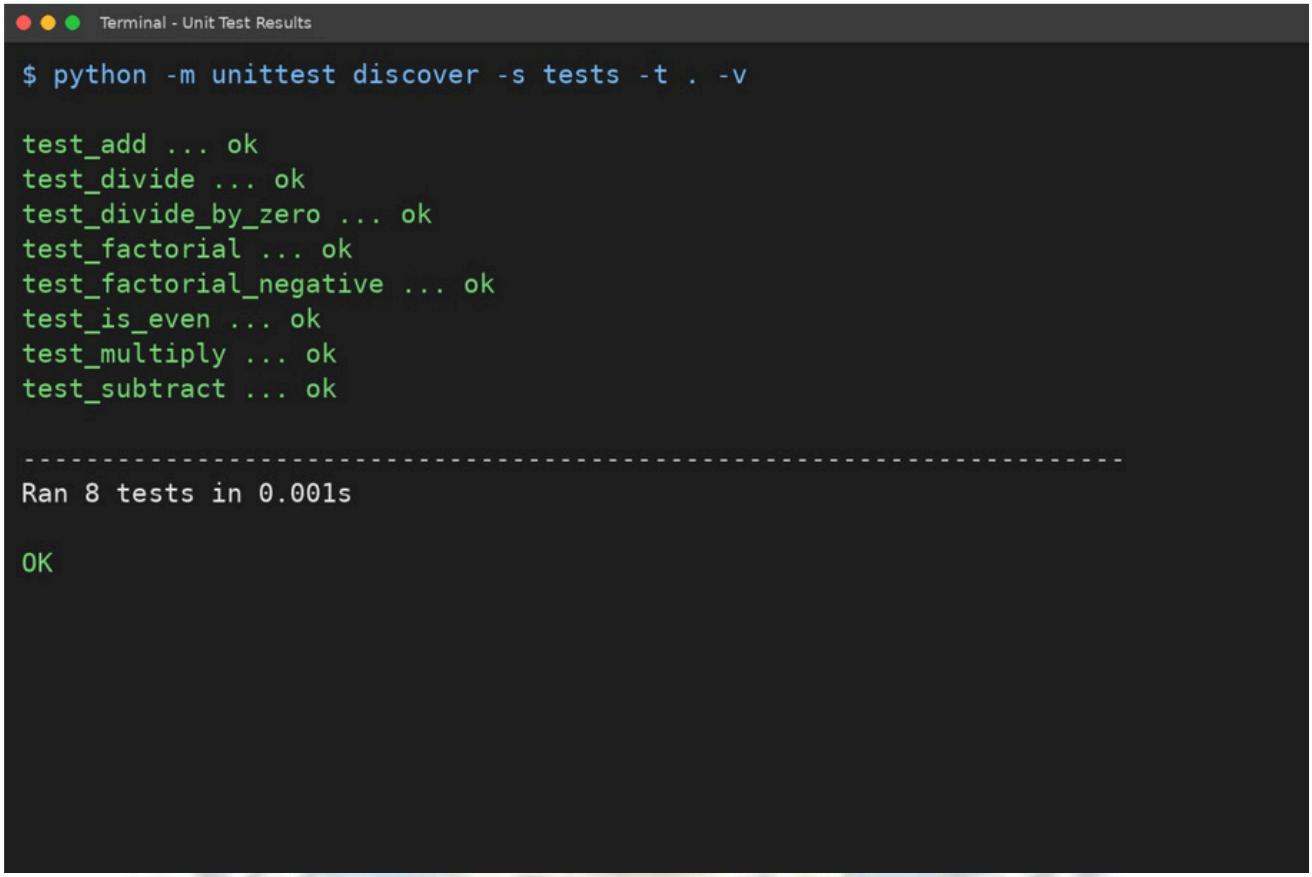


FIG:1

UGC AUTONOMOUS

.github/workflows/ci.yml

```yaml
name: Unit Testing CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Run unit tests
        run: |
          python -m unittest discover \
            -s tests -t . -v

      - name: Build Docker image
        run: docker build -t unit-test-app .
```

FIG:2

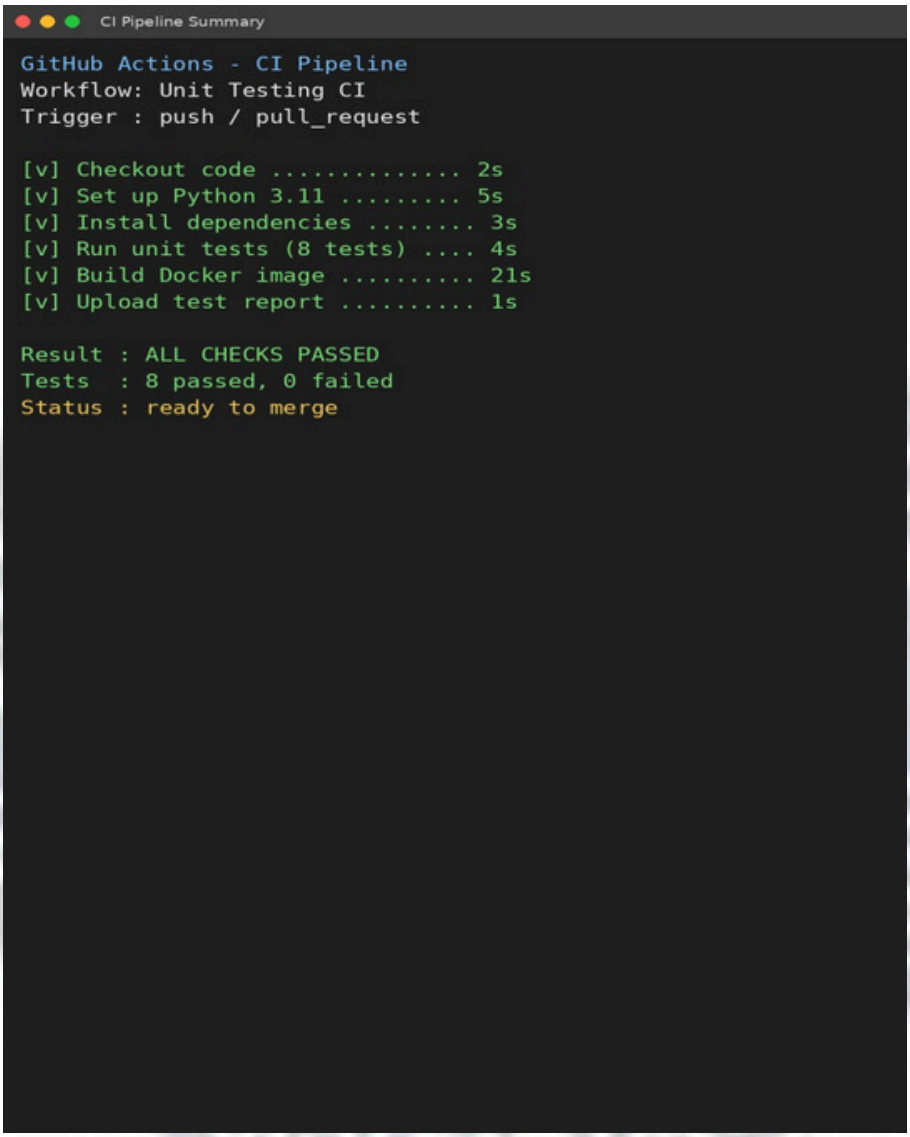

CI Pipeline Summary
GitHub Actions - CI Pipeline
Workflow: Unit Testing CI
Trigger : push / pull_request
[v] Checkout code 2s
[v] Set up Python 3.11 ..... 5s
[v] Install dependencies 3s
[v] Run unit tests (8 tests) 4s
[v] Build Docker image 21s
[v] Upload test report 1s
Result : ALL CHECKS PASSED
Tests : 8 passed, 0 failed
Status : ready to merge

FIG:3

9. CONCLUSION

Unit Testing using DevOps shows how automated tests can be built into the software delivery process. The calculator application was tested with 8 unit tests that cover normal inputs, boundary values, and error conditions such as division by zero and negative factorial. Each test was executed automatically by the CI pipeline whenever code was pushed, and all tests passed. The project demonstrates that automated unit testing finds defects early, reduces manual effort, gives developers quick feedback, and prevents faulty code from being deployed. Using Git for version control and Docker for a consistent environment makes the process repeatable on any machine. Overall, combining unit testing with DevOps practices leads to higher quality software and faster, safer releases.

UGC AUTONOMOUS

10. FUTURE ENHANCEMENTS

In the future, this project can be extended with code coverage reports so that untested parts of the code are easy to identify. Integration, system, and performance tests can be added to the pipeline after the unit tests. Static code analysis and security scanning tools such as SonarQube can be included to check code quality and vulnerabilities. The pipeline can also be extended to Continuous Deployment, automatically releasing the Docker image to a cloud platform using Kubernetes, with monitoring tools such as Prometheus and Grafana to track the application after release. Notifications through email or Slack can be added so the team is alerted immediately when a build fails.

11. REFERENCES

- https://docs.python.org/3/library/unittest.html
- https://docs.github.com/en/actions
- https://docs.docker.com/get-started/
- https://git-scm.com/doc
- https://www.jenkins.io/doc/
- https://www.atlassian.com/devops
- https://martinfowler.com/articles/continuousIntegration.html

